

# 第五章 单元测试与集成测试

- 5.1 代码评审与分析
  - 评审方式（P106）
- 5.2 代码评审案例分析（P110）
- 5.3 代码静态分析工具
  - ▣ SonarLint
  - ▣ Java: FindBugs、PMD、Checkstyle等
  - ▣ C/C++: Clint/Oclint/Cpplint、TestC++
  - ▣ Python: Pylint、PySonar2等
  - ▣ 安全: Coverity、Fortify等
  - 代码分析工具常见的使用场景

## 使用方式

在IDE中实时进行分析  
代码每次提交时进行  
与CI/CD流水线集成

## 使用目的

为开发者快速返回检查结果  
及时消除常见的安全问题

## 规则集

可选有限个检测项

## 检测耗时

几秒~几分钟

## 检测项举例

- SQL注入
- 跨站脚本攻击
- 资源泄露
- 硬编码凭据
- 配置审查

- 5.4 单元测试的目标和任务

## 单元测试(Unit Testing)

单元测试是对软件组成单元进行测试。其目的是检验软件基本组成单位的正确性。测试的对象是软件设计的最小单位：模块。又称为模块测试

测试阶段：编码后或者编码前（TDD）

测试对象：最小模块

测试人员：白盒测试工程师或开发工程师

测试依据：代码和注释+详细设计文档

测试方法：白盒测试

测试内容：模块接口测试、局部数据结构测试、路径测试、错误处理测试、边界测试

- 单元测试(Unit Testing)是指对软件中的最小可测试单元或基本组成单元进行检查和验证。
- 尽早发现错误，成本越低。单元质量是系统质量的基石，单元测试可以比系统测试测得更彻底。
- 单元测试的目标
  - ▣ 单元模块被正确实现（主要指编码），包括功能、性能、安全性等，但一般主要介绍单元功能测试。
  - ▣ （参数）输入是否正确传递和得到保护（容错），输出是否正常
  - ▣ 内部数据能否保持其完整性，包括变量的正确定义与引用、内存及时释放、全局变量的正确处理和影响最低
  - ▣ 代码行、分支覆盖或MC/DC达到要求，如高于80%或95%
- 驱动程序与桩程序（P124）
  - 驱动模块(Driver)
    - 模拟被测单元的上级模块，能调用被测模块。
    - 接收测试数据、调用被测模块和并把相关的数据传送给被测模块。
    - 作为入口，可以设置不同的数据参数来完成各种测试用例。
  - 桩模块(Stub)
    - 模拟被测模块所调用的下层模块。
    - 桩模块由被测模块调用，一般只进行很少的数据处理（如打印入口和返回），以便检测被测模块和下级模块的接口。
  - 示例

```

1 //Program 1
2 #include <iostream.h>
3 void get_input(x& cost, int& y);
4 int main( ) ←----- Driver
5 {
6     double a;
7     int b;
8     char ans;
9     do
10    {
11        get_input(a, b); ←----- Function
12        cout.setf(ios::fixed);          under
13        cout.setf(ios::showpoint);      test
14        cout.precision(2);
15        cout << "a is " << a << endl;
16        cout << "b is " << b << endl;
17
18        cout << "Test again?"
19              << " (Type y for yes or n for no): ";
20        cin >> ans;
21        cout << endl;
22    } while (ans == 'y' || ans == 'Y');
23    return 0;
24 }
25 //Program 2
26 void function_under_test(int& x, int& y) {
27     ...
28     p = price(x);
29     ...
30 }
31 double price(int x) {return 10.00;} ←----- Stub

```

## • 5.6 单元测试工具

- 有哪些：JUnit, CppUnit, NUnit (C#的), pytest, unittest, GoogleTest。

### • JUnit

- JUnit的关键组件

#### • Assert类

Assert（断言）类是JUnit框架中非常重要的类，用于判断所返回的实际结果与预期结果是否相符。

#### • Annotation（注解）

JUnit中以@引导标注测试类中不同方法的功能。

#### • Runner（运行器）

JUnit通过运行器Runner和显示测试执行结果。

- 常见注解（各个注解的详细解释见课本P134）

✓ **Annotation** 清晰地表达测试程序的逻辑结构和功能

✓ 常用的JUnit **Annotation** 包括@Test、@ParameterizedTest、@BeforeEach、@AfterEach、@BeforeAll、@AfterAll、@RepeatedTest、@TestFactory、@TestTemplate、@TestClassOrder、@TestMethodOrder、@DisplayName、@Nested、@Tag、@Disabled、@Timeout、@ExtendWith、@RegisterExtension、@TempDir

- [@Test](#)：表明这是一个测试方法。

- 对于方法的声明的要求：名字可以随便取，没有任何限制，但是返回值必须为void，而且不能有任何参数。如果违反这些规定，会在运行时抛出一个异常。

- @Test注解防止超时的处理方式

给 **@Test** 标注加一个参数即可

```
@Test(timeout=1000 ) //1000毫秒
public void testsquareRoot() {
    calculator.squareRoot( 4 );
    assertEquals( 2 , calculator.getResult());
}
```

- [@BeforeClass](#) 和 [@AfterClass](#)

- 只在测试用例初始化时执行 [@BeforeClass](#) 方法，当所有测试执行完毕之后，执行 [@AfterClass](#) 进行收尾工作。
  - 注意：每个测试类只能有一个方法被标注为 [@BeforeClass](#) 或 [@AfterClass](#)，并且该方法必须是 Public 和 Static 的。
  - 使用示例：有一个类负责对大文件进行读写，在调用每一个方法之前，我们都要打开一个大文件并读入文件内容，这是一个非常耗费时间的操作。可以用 [@BeforeClass](#) 和 [@AfterClass](#) 注解实现在执行所有测试用例之前打开大文件，执行完毕后关闭的工作。

- Assert类

- 检查并判断程序的运行结果是软件测试中的一项重要工作。测试程序必然会包含一些断言条件，程序运行时不满足这些条件，则表明程序的运行状态/结果与期望不一致，程序中存在缺陷。
  - JUnit通过Assert类提供了一系列断言方法来判断程序的运行结果。
  - 断言方法中的参数包括期望变量expected 和实际变量actual。相等即成功，相异即失败。
  - 断言方法（P134）

**1, assertEquals([String message],expected,actual)**

**message:** 可选，假如提供，将会在发生错误时报告这个消息

**expected:** 是期望值，通常都是用户指定的内容。

**actual:** 是被测试的代码返回的实际值。

例: assertEquals("equals","1","1");

**2, assertEquals([String message],expected,actual,tolerance)**

**message:** 是个可选的消息，假如提供，将会在发生错误时报告这个消息。

**expected:** 是期望值，通常都是用户指定的内容。

**actual:** 是被测试的代码返回的实际值。

**tolerance:** 是误差参数，参加比较的两个浮点数在这个误差之内则会被认为是相等的。

例: assertEquals ("yes",6.6,13.0/2.0,0.5);

**3, assertTrue ([String message],Boolean condition)**

**message:**是个可选的消息，假如提供，将会在发生错误时报告这个消息。

**condition:**是待验证的布尔型值。

该断言用来验证给定的布尔型值是否为真，假如结果为假，则验证失败。

例: assertTrue("true",0==0);

**4, assertFalse([String message],Boolean condition)**

**message:**是个可选的消息，假如提供，将会在发生错误时报告这个消息。

**condition:**是待验证的布尔型值。

该断言用来验证给定的布尔型值是否为假，假如结果为真，则验证失败。

例: assertFalse("false",4==5);

**5, assertNull([String message],Object object)**

**message:**是个可选的消息，假如提供，将会在发生错误时报告这个消息。

**object:**是待验证的对象。

该断言用来验证给定的对象是否为null，假如不为null，则验证失败。

例: assertNull("null",null);

**6, assertNotNull([String message],Object object)**

**message:**是个可选的消息，假如提供，将会在发生错误时报告这个消息。

**object:**是待验证的对象。

该断言用来验证给定的对象是否为null，假如不为null，则验证失败。

该断言用来验证给定的对象是否为非null，假如为null，则验证失败。

例: assertNotNull("not null",new String());

**7, assertEquals ([String message], expected,actual)**

**message:**是个可选的消息，假如提供，将会在发生错误时报告这个消息。

**expected:**是期望值。

**actual:**是被测试的代码返回的实际值。

该断言用来验证expected参数和actual参数所引用的是否是同一个对象，假如不是，则验证失败。

例: assertEquals("same",6,3+3);

**8, assertNotSame ([String message], expected,actual)**

**message:**是个可选的消息，假如提供，将会在发生错误时报告这个消息。

**expected:**是期望值。

**actual:**是被测试的代码返回的实际值。

该断言用来验证expected参数和actual参数所引用的是否是不同对象，假如所引用的对象相同，则验证失败。

例: assertNotSame("not same",5,4+2);

**9, fail([String message])**

**message**是个可选的消息，假如提供，将会在发生错误时报告这个消息。

该断言会使测试立即失败，通常用在测试不能达到的分支上（如异常）。

➤ **TestCase类**是JUnit框架中**为测试用例提供服务的**核心类，同样位于junit.framework包中，并继承自Assert类，因此可直接使用Assert类中的相关方法。单元测试所编写的测试类均需直接或间接继承于TestCase类，依靠其提供的方法来实现测试用例（@Test所注解的测试方法）的运行与判断。

➤ **TestSuite类**实现了JUnit Test接口，用于管理JUnit中的每一个测试用例

➤ **TestResult类**收集并记录所有测试用例的运行结果。测试用例的运行结果可分为成功、失败、错误等三类

- Junit5示例

```
class StandardTests {

    @BeforeAll
    static void initAll() {
    }

    @BeforeEach
    void init() {
    }

    @Test
    void succeedingTest() {
    }

    @Test
    void failingTest() {
        fail("a failing test");
    }

    @Test
    @Disabled("for demonstration purposes")
    void skippedTest() {
        // not executed
    }

    @Test
    void abortedTest() {
        assertTrue("abc".contains("Z"));
        fail("test should have been aborted");
    }

    @AfterEach
    void tearDown() {
    }

    @AfterAll
    static void tearDownAll() {
    }

}
```

- Mock技术

- 相关概念



- Fake（伪对象）
  - 定义：几乎和产品代码做同样的工作（伪装成跟真的一样），但为了满足测试要求，更简单地实现。
  - 用途：通常用于数据库或网络连接的测试，提供一个内存中的替代实现。
  - 例子：一个内存数据库实现，可以代替实际的数据库连接。如要验证DAO (Data Access Object)能否正常工作，就构造fake object代替数据库。
- Dummy（哑对象）
  - 定义：Dummy对象是完全不使用的对象，只用于满足方法的签名。它们通常不会被方法调用真正使用。
  - 用途：用于填充参数，以便可以调用一个方法或构造一个对象。
  - 例子：传递给方法的对象，但该对象在方法中没有实际使用，如一个空对象或默认构造对象。
- Double（测试替身）
  - 定义：测试替身是一个总称，包含所有替代真实对象的各种类型（如Dummy, Fake, Spy, Mock）。它们在测试中替代真实对象以模拟其行为。
  - 用途：用于任何需要替代真实对象的情况，具体使用何种类型的替身取决于测试需求。
  - 例子：包含Fake, Dummy, Spy, Mock等。
- Spy（间谍对象）
  - 定义：Spy对象在一定程度上是实际对象的替代品，并可以记录实际对象的交互情况。Spy对象既能像真实对象一样工作，又能记录下方法调用、参数和返回值等。
  - 用途：用于验证对象的某些方法是否被调用、调用次数、参数等，通常在测试行为和交互时使用。
  - 例子：一个Spy对象可以跟踪某个方法被调用了几次以及使用了哪些参数。
- Mock（模拟对象）
  - 定义：Mock对象是严格按照预期行为进行验证的对象。它们不仅用于模拟实际对象的行为，还用于验证某些交互是否按预期发生。Mock对象预先定义了某些行为和期望，并在测试过程中检查这些期望是否被满足。
  - 用途：用于测试对象之间的交互，确保某些方法在特定条件下被调用，并且调用时使用了预期的参数。
  - 特点：Mock对象通常是由Mock框架（如Mockito、EasyMock等）生成的，它们可以预先设定预期行为，并在测试结束时验证这些行为是否被满足。
- Stub和mock小结

- ▣ Stub最弱，只是参与测试，为了让测试跑起来
- ▣ Dummy/Spy 强些，Test Spy 参与测试，还要验证产生的某种结果。  
Dummy是简单、原始的测试替身，如Dummy server，给它不同的输出会有不同的相应，但其内部没有实质计算，是接口或基类的衍生
- ▣ Mock更强，要求如何参与，且比较灵活，视情况而定，其行为可能会像Dummy 、Stub 或 Spy
- ▣ 够用就行，如果Stub够用，就不要用Mock

- Mock框架

- JMock
- EasyMock
- Mockito（目前主流的支持Java语言的Mock框架）
- Moco（Java API、独立服务器）

- 测试覆盖率工具

- 代码覆盖率分析工具

- ▣ JaCoCo通过对编译后的Java class文件进行统计代码的插装，测试覆盖率的收集与报告。
- ▣ BullseyeCoverage是一种获取C/C++的代码覆盖率的工具，支持分支覆盖率（Branch Coverage）的分析
- ▣ **Testwell CTC++**, Parasoft Jtest/C++Test, CoverageMeter, Jcov, CodeCover, Coverage.py.....

- 5.7 持续集成测试

- 集成测试

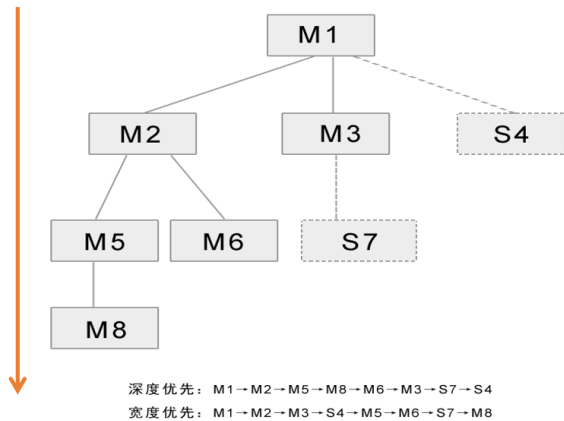
- **集成测试(Integration Testing)**

集成测试也称联合测试（联调）、组装测试，将程序模块采用适当的集成策略组装起来，对系统的接口及集成后的功能进行正确性检测的测试工作。集成主要目的是检查软件单位之间的接口是否正确。

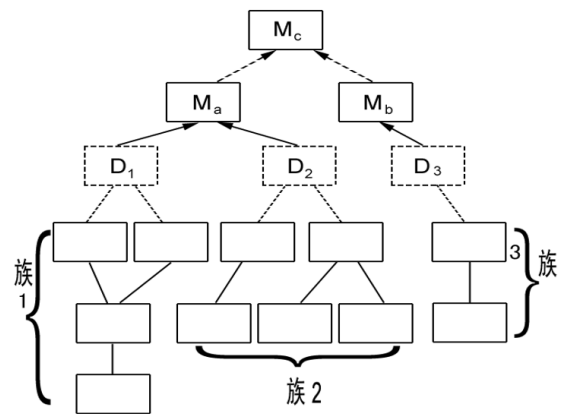
- 测试阶段：一般单元测试之后进行
- 测试对象：模块间的接口
- 测试人员：白盒测试工程师或开发工程师
- 测试依据：单元测试的模块+概要设计文档
- 测试方法：黑盒测试与白盒测试相结合
- 测试内容：模块之间数据传输、模块之间功能冲突、模块组装功能正确性、全局数据结构、单模块缺陷对系统的影响

- 单体架构的集成测试

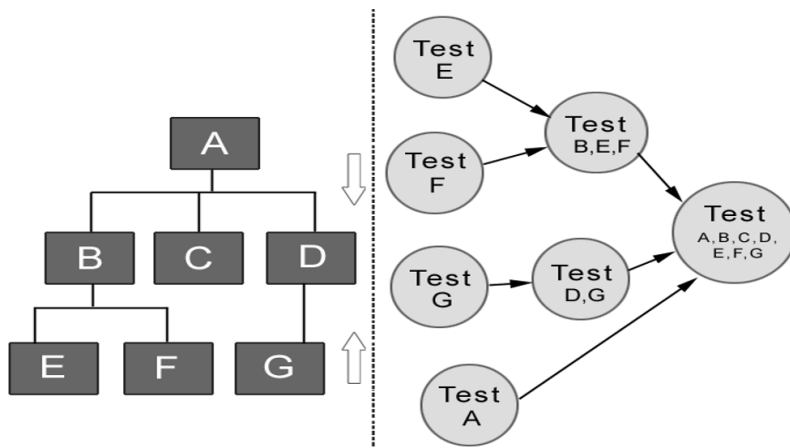




自顶向下

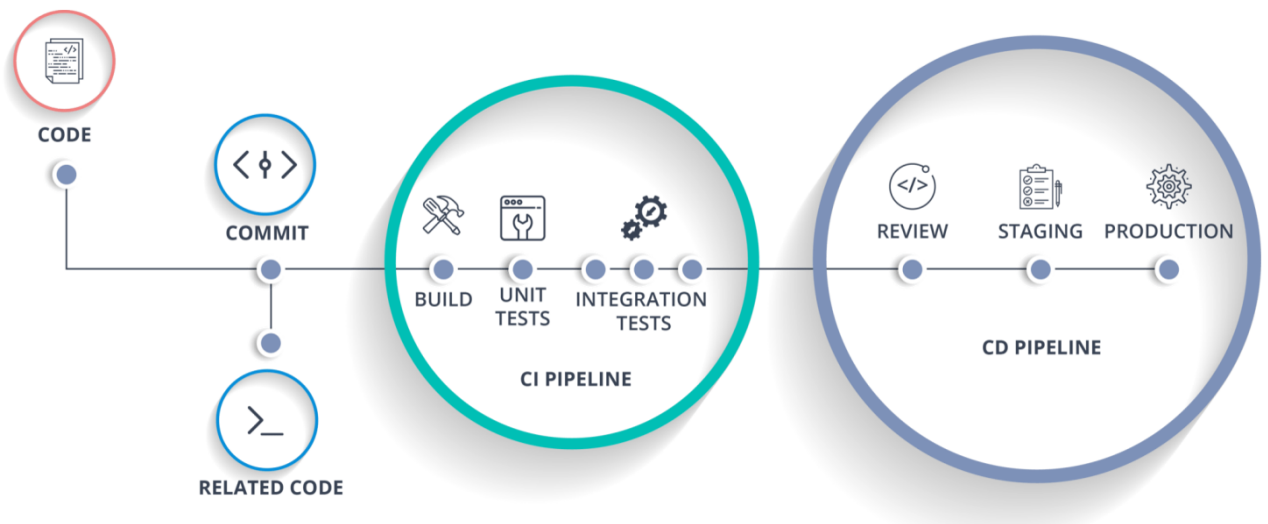


自底向上

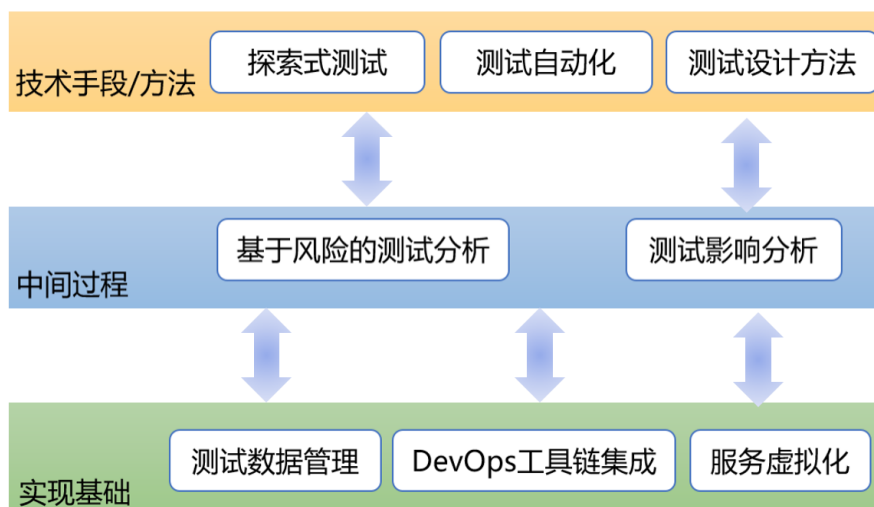


混合策略

- 微服务架构的集成测试
- CI/CD与持续测试



## 持续测试实践框架



### 持续测试的特点

- 足够快
- 准确且有效
- 平滑有序
- 与CI/CD集成

整个测试过程要快，一方面高度自动化测试，另一方面业务端到端的探索式测试

被测系统往往很复杂，不可能做全回归测试，而是要推行精准测试

打通整个测试过程：测试左移到测试右移、单元测试到系统测试、静态测试到动态测试

与研发的持续构建、持续集成、持续部署、运维等环境的集成

### 小结

## 本章小结

- 深刻理解代码规范和代码评审的价值，持续进行代码评审，采用互为评审、集体评审，团队有多方面的收益
- 代码分析工具的使用，今天很流行，包括开发本地扫描代码、与CI/CD流水线集成等。
- 深刻理解单元测试（UT）的价值和xUnit框架实现的机制，重视单元测试，掌握JUnit这类工具和mock工具的使用技巧
- 单元测试是否充分，需要衡量代码语句/分支覆盖率，一般会使用覆盖率工具进行自动分析
- 今天提倡持续测试，以促进持续交付；契约测试值得关注，从开发自测开始，比较彻底保证质量